Week 44, Tuesday Session

Flipped classroom: cross-validation for your Project 2 network – half an hour of theory, then the six questions of exercise 1, then your own network inside `scikit-learn`

Companion notebook: `week44tuesday.ipynb`

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

How today works

Part A (about 30 minutes, a regular lecture)

The theory of cross-validation in one pass: three errors and three data sets; the k-fold estimate, its standard error and the one-standard-error rule; leakage and why the scaler lives inside the fold; the hyperparameters of a network, epochs among them; what `scikit-learn` provides.

Case 1 (about 20 minutes): the six questions

Exercise 1 of `exercisesweek44.ipynb`, in groups, pencil and paper; then the answers in plenum, one at a time.

Case 2 (about 35 minutes): your network inside `scikit-learn`

Exercise 2: wrap your own network as an estimator, then let `KFold`, `cross_validate`, `GridSearchCV` and `validation_curve` do the folds, the scaling inside the folds, the grid and the bookkeeping. Six steps, checkpoints after each; step 4 (width, epochs) is the optional part.

Last five minutes

What to finish before Friday; a mention – only a mention – of how the same is done with PyTorch and Keras; what next Monday builds on.

What you need, and what today is for

What you need

Your Project 2 network (or the week-41 code: `create_layers`, `train`, `predict`), `scikit-learn`, and the data cell of Project 2 part b) – 100 points of the Runge function with noise $\sigma = 0.1$, seed 2026. The notebook has everything, with the week-41 network filled in as the estimator.

What today is for – and what is required

Every number you report on the test set must come from a model whose every choice was made on training data alone – today is how. **Required**, in exercise 2 and in Project 2: cross-validate the learning rate γ and the penalty λ in front of the L2 or L1 norm (the heat map of Case 2, step 3).

What is optional

Cross-validating the width, the depth, the activation and the number of epochs (Case 2, step 4; exercise 2 c–d) – optional in the exercises and in Project 2, shown today so that you know how; the machinery is the same. Writing the cross-validation loop yourself (exercise 4). Understanding what the library does is not optional.

Part A.1: two kinds of unknowns

Parameters

The weights and biases of the network (or $\boldsymbol{\theta}$ of a linear model). For given hyperparameters they are fixed by *minimising the cost on the training data*.

Hyperparameters

The penalty λ, the learning rate γ, the width and depth, the activation, the batch size, the number of epochs. They cannot be fixed by the same minimisation, and the reason is exact.

Why the training error cannot choose them (3.91)

The training error is non-decreasing in λ along the whole path,

$$\mathrm{Err}_{\mathrm{Train}}(\lambda) \;\geq\; \mathrm{Err}_{\mathrm{Train}}(0),$$

so minimising it over λ always returns $\lambda = 0$. It *falls* with every hidden unit and every epoch, so minimising it over those returns the widest network trained longest. The training error chooses nothing.

Part A.2: three errors, three data sets

Three errors, kept apart

1. the **training error**: minimised over the parameters, at fixed hyperparameters;
2. the **cross-validation error**: minimised over the hyperparameters;
3. the **test error**: computed *once*, at the very end, minimised over nothing.

Three data sets

- the **test set**: set aside first, not looked at until the last step;
- the **training data**: everything else – scaling, folds, grid and refit all happen inside it;
- the **validation fold**: a piece of the training data that plays the test set's role while choices are made.

k-fold cross-validation lets every training point play the validation role exactly once. That is the whole idea; the rest of Part A is the details that make it correct.

Part A.3: the recipe of Section 3.14, which Case 2 follows

1. Set aside a test set and do not look at it again until step 6.
2. Choose a grid of hyperparameter values – logarithmic for γ and λ.
3. Shuffle the training data and split it into K folds, K between five and ten.
4. For every fold and every grid point: standardise with the *training folds'* statistics, fit on the training folds, predict the held-out fold, record the error. Average to obtain the CV error and its standard error.
5. Select the minimum, or the one-standard-error choice, and *refit on all the training data* at that value.
6. Evaluate the refitted model on the test set, once. Report the selected hyperparameters, the CV error with its standard error, and the test error.

Six steps, six places where something can go wrong. Part A takes them in turn.

Part A.3, an example: a question from the group sessions

The question

"I have 100 points. I set 20 aside as a test set, use 20 as a validation set and train on the remaining 60. Once I have chosen the hyperparameters, may I refit on all 80 non-test points – the validation points included?"

Part A.3, an example: a question from the group sessions

The question

"I have 100 points. I set 20 aside as a test set, use 20 as a validation set and train on the remaining 60. Once I have chosen the hyperparameters, may I refit on all 80 non-test points – the validation points included?"

The answer: yes

The validation points were used to *select* the hyperparameters; that job is finished. Fit a fresh model on all 80 non-test points with the selected hyperparameters, then evaluate it *once* on the 20 untouched test points. The test set is still independent: nothing about the model was chosen on it.

Part A.3, an example: a question from the group sessions

The question

"I have 100 points. I set 20 aside as a test set, use 20 as a validation set and train on the remaining 60. Once I have chosen the hyperparameters, may I refit on all 80 non-test points – the validation points included?"

The answer: yes

The validation points were used to *select* the hyperparameters; that job is finished. Fit a fresh model on all 80 non-test points with the selected hyperparameters, then evaluate it *once* on the 20 untouched test points. The test set is still independent: nothing about the model was chosen on it.

What that single split is, and what it is not

One fixed 60/20 split is **holdout validation**: one validation error, from one set of 20 points, with no error bar. The same numbers give a natural **four-fold cross-validation**: cut the 80 non-test points into four folds of 20 and let the validation set *rotate*, so that every one of the 80 points is validated exactly once – next frame.

Part A.3, the example continued: four folds of twenty

For every candidate hyperparameter setting, four fits

Fit	Training data	Validation data
1	folds 2, 3, 4 (60 pts)	fold 1 (20 pts)
2	folds 1, 3, 4 (60 pts)	fold 2 (20 pts)
3	folds 1, 2, 4 (60 pts)	fold 3 (20 pts)
4	folds 1, 2, 3 (60 pts)	fold 4 (20 pts)

The six steps, on these numbers

1. Set aside 20 points for testing; keep them out of model selection.
2. Divide the remaining 80 into four folds of 20.
3. For each candidate setting, perform the four fits of the table.
4. Choose the setting with the best *average* validation performance over the four folds (their spread is the error bar).
5. Refit on all 80 points with the selected setting.
6. Evaluate once on the 20 test points.

Two practical details that matter

- **Preprocessing belongs inside the procedure.** In each fold, fit the scaling (imputation, feature selection) on its 60 training points only; for the final model, refit it on all 80 – what a `Pipeline` does for you (A.12).
- **Keep the hyperparameters fixed after looking at the test result.** Change the model in response to it, and the test set has become another validation set.

Part A.4: the k-fold estimate (Section 2.12, Eq. 3.92)

Shuffle; cut into K folds of equal size; let $\kappa(i)$ be the fold of observation i. For each fold k: hold it out, fit on the other $K-1$, predict the held-out fold, discard the model.

The cross-validation error

The average error of every observation predicted by the model that *did not see it*:

$$\mathrm{CV}_K = \frac{1}{n}\sum_{i=0}^{n-1}\left(y_i - \hat{f}^{(-\kappa(i))}(\mathrm{x}_i)\right)^2 = \frac{1}{K}\sum_{k=1}^{K}\mathrm{MSE}_k.$$

Every observation is validated exactly once and trained on $K-1$ times. The K fold models are temporary: averaged, then thrown away.

Part A.5: what the estimate is *of* – read this twice

The statement

CV_K estimates the expected error on unseen data of the **training procedure** – this architecture, these hyperparameters, this optimiser – applied to *training sets of size* $n(K-1)/K$.

What it is not

Not the error of the one network you finally train on all n points. That network is a *new* fit, which cross-validation never evaluated.

Why the estimate is still the right one to use

The final model is the same recipe applied to slightly more data, so the CV error is the best available estimate of its error – slightly pessimistic, because the fold models had n/K fewer points.

Part A.6: choosing K, from 2 to n

Small K

The fold models train on much less than n points: the estimate is *biased upwards* (pessimistic). For $K = 2$ the procedure is judged on half the data.

$K = n$, leave-one-out

No bias from the training-set size and no randomness from the fold assignment – it is the jackknife of Eq. (2.53) applied to prediction. But n fits, and the n training sets are almost identical, so the fold errors are strongly correlated and the estimate has *high variance*.

The compromise

$K = 5$ or $K = 10$.

What it costs for a network

K folds $\times$ every grid point $\times$ every epoch. Case 2 has $5 \times 25 \times 1000$ epochs on 80 points – a minute. On MNIST, exercise 3 prescribes a 10 000-image subset for a reason, and random search over the grid (Section 8.12) is often the better use of the budget.

Classification

Cut *within each class*, so that every fold carries the class proportions – `StratifiedKFold`. Otherwise a rare class may be missing from a training fold entirely.

Part A.7: the standard error, and why it is optimistic (Eq. 3.93)

The spread of the K fold errors

$$\mathrm{SE}_K = \frac{1}{\sqrt{K}} \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} \left(\mathrm{MSE}_k - \mathrm{CV}_K\right)^2}$$ – Eq. (2.24) applied to K numbers.

A rough guide, not a confidence interval

Two training sets of a K-fold share a fraction $(K-2)/(K-1)$ of their points – 3/4 for $K = 5$. The fold errors are therefore positively correlated, and (3.93) *understates* the true uncertainty.

A library detail that bites

`GridSearchCV` reports `std_test_score` with `ddof=0`. The standard error of (3.93) is `std_test_score / np.sqrt(K - 1)`. And every score is a *negative* MSE, because the library maximises.

Part A.8: the one-standard-error rule (Eq. 3.96)

The problem

Near its minimum the CV curve is flat, and the position of the minimum is known only to within the noise of the estimate.

The rule

Among the hyperparameters whose CV errors are within one standard error of the best, take the *simplest*:

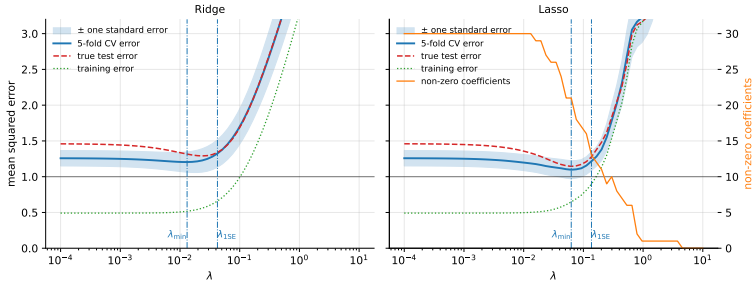
$$\lambda_{1\mathrm{SE}} = \max\bigl\{\lambda : \ \mathrm{CV}_K(\lambda) \le \mathrm{CV}_K(\lambda_{\min}) + \mathrm{SE}_K(\lambda_{\min})\bigr\}.$$

The improvement offered by the more complex model is not larger than the uncertainty with which it is measured, so it is not believed. A convention, not a theorem – and it errs deliberately towards simplicity.

"Simplest" for a network

Largest λ; *fewest* hidden units; *fewest* epochs. The learning rate affects the optimiser, not the model, so on a two-axis grid apply the rule along the regularisation axis.

Part A.9: book Figure 3.4, from week 36 – the picture does not change



The CV curve with its standard errors, the minimum, the one-SE choice, and (dashed) the true test error of the refit, for Ridge and the Lasso on a sparse truth. Today the model is a neural network and the axis may be the learning rate, the width or the number of epochs; the picture, and the way to read it, are the same.

Part A.10: then refit

The final model

A *new* fit at the chosen hyperparameters on all n training points, scaled with the statistics of all n. In `scikit-learn`: `best_estimator_`, produced by `refit=True`, the default of `GridSearchCV`.

Which λ is being transferred (3.95)

The fold models were fitted to $n(K-1)/K$ points, the final model to n. With the cost written *per observation*, $\frac{1}{n}\|y - X\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2$ – the convention of Project 2 and of our network – a λ found on 80 points is the right λ for 100.

Only then

The test set, once.

Part A.11: leakage – the held-out fold is future data

The rule

Nothing computed from the held-out fold may enter the fit. Any preprocessing that *learns* from the data – centring, scaling, feature selection, imputation, PCA – is fitted inside each fold, on the training part only, and applied frozen to the held-out part.

What leaks if you scale first

Standardise the whole training set once, and every training fold has seen the *mean and variance of its own validation fold*. For scaling alone the leak is small. For anything that looks at the targets it is large: screen the features by their correlation with y on the whole set, cross-validate the survivors, and the CV error can fall below the noise floor for features that are pure noise (Section 3.14).

For a network

The same for the inputs; and the early-stopping epoch (next frame) must be chosen on a validation fold, never on the fold being scored.

Part A.12: the mechanism – a `Pipeline`

```
net = make_pipeline(StandardScaler(), YourNet())
```

- Cross-validate the *pipeline*, not the network.
- On every fold the library calls `fit` on the pipeline: it fits the scaler on the training part, transforms it, fits the network on the result.
- At prediction time it transforms the held-out part with the *stored* training statistics.
- Hyperparameters are addressed through the step name: `yournet__gamma`, `yournet__lmbd` (two underscores; the step name is the class name in lower case; `net.get_params().keys()` lists them all).

Nothing else enforces the rule of the previous frame. The pipeline is how you make leakage impossible rather than merely avoided.

Part A.13: the hyperparameters of a network (Section 8.12)

The list

Learning rate γ and the optimiser's own knobs; penalty λ (L2, L1); width and depth; activation; batch size; **number of epochs**; initialisation seed; dropout rate.

Two facts about all of them, and what we ask of you

None can be chosen by the training error. All of them change the test error, and strongly: in Case 2 the CV error moves by a factor of four across the γ grid alone. Required in exercise 2 and Project 2: γ and λ. Optional: width, depth, activation, epochs – same tools, your choice.

Epochs are a hyperparameter: early stopping

The training error falls with every epoch and says nothing. The validation error falls, flattens and – for a network that can overfit – rises again; the epoch at which it is smallest is the chosen number of epochs. Two ways with the library: a `validation_curve` over `epochs` (Case 2, step 4; a full training per value), or an estimator that records the validation error *during* one training run and keeps the best weights (what Keras' `EarlyStopping` does; exercise 2d invites you to build one).

Part A.14: validation curves, and the optimism of the best score

Validation curves

One hyperparameter at a time, the others fixed: CV error and training error against the value, with the standard error. Where the two curves separate the model begins to overfit; where the CV curve is flat the data cannot tell the values apart. The figures parts c)–d) of Project 2 ask for are exactly these.

The best CV score of a grid is optimistic

The minimum of 25 noisy estimates is biased *downwards*: you chose it because it was low, and part of why it was low is noise. The CV error of the winner is therefore *not* an unbiased estimate of its test error. That is what the test set, touched once at the end, is for – or nested cross-validation, an outer loop around the whole selection, when there is no test set to spare.

Reporting

The chosen hyperparameters and how they were chosen; the CV error $\pm$ standard error *and* the training error beside it; the test error, once. "The best learning rate is 0.01, test MSE 0.0094" reports a choice made on the test set – Case 1, question d).

Part A.15: what `scikit-learn` provides

theory	library
the folds	`KFold(k, shuffle=True, random_state)`
stratified folds	`StratifiedKFold`
the K fold errors, (3.92)	`cross_val_score`
plus training errors and fit times	`cross_validate`
the grid, (3.94), and the refit	`GridSearchCV`, `best_estimator_`
every number of the grid	`cv_results_`
one hyperparameter at a time	`validation_curve`
error against training-set size	`learning_curve`
out-of-fold predictions	`cross_val_predict`
scaling inside the fold	`Pipeline`

What they do *not* do: choose the grid, apply the one-standard-error rule (`cv_results_` has what you need), or stop you from calling `fit` on the test set.

Part A.16: the three rules of an estimator – all the library asks of your code

1. `__init__` stores its keyword arguments *under the same names* and does nothing else. `BaseEstimator` then supplies `get_params`/`set_params` – how `GridSearchCV` sets γ and λ – and `clone` gives every fold a fresh, untrained copy.
2. `fit(X, y)` does all the work, stores the result in attributes ending in an underscore (`self.layers_`), and *returns `self`*.
3. `predict(X)` returns a 1D array; `RegressorMixin` supplies `score` (the R^2). A classifier adds `predict_proba` and `classes_`.

```
class NeuralNetRegressor(BaseEstimator,
      RegressorMixin):
    def __init__(self, hidden=(50,), activation="
         sigmoid",
                gamma=0.03, lmbd=0.0, epochs=1000,
                batch_size=10, seed=2026):
        self.hidden = hidden
        self.activation = activation
        self.gamma = gamma
        self.lmbd = lmbd
        self.epochs = epochs
        self.batch_size = batch_size
        self.seed = seed

    def fit(self, X, y):
        ... # your training code; store self.layers_
        return self

    def predict(self, X):
        ... # your forward pass, shape (len(X),)
```

Case 1: the six questions of exercise 1 – pencil and paper, in groups

a) What is estimated

In one sentence: the k-fold CV error estimates the error of *which* model, trained on how much data, evaluated on what? Why is it about the training *procedure* rather than the one network you train at the end?

b) The mean and its error

Give $\bar{e} \pm s/\sqrt{k}$ for the k fold errors. Why are the k numbers not independent (what fraction do two training sets share?), and what does that do to the standard error? Bias and variance of the estimate as k goes from 2 to n?

c) Scaling

Why must the scaler be fitted *inside* each fold? Name one number that leaks otherwise. How does a `Pipeline` guarantee it, in one sentence?

Case 1: the six questions, continued

d) The colleague

"Ten networks, ten learning rates, the test MSE of each; the best learning rate is 0.01, with a test MSE of 0.0094." Two things are wrong. Write the honest version.

e) Early stopping

Why is the number of epochs a hyperparameter like any other? How is a validation fold used to choose it, and why is the training error useless for that?

f) The one-standard-error rule

State it for $\lambda \in \{0, 10^{-5}, \ldots, 10^{-2}\}$. Why does it prefer the *more* regularised model among those within one standard error – and what does "more regularised" mean when the grid is over hidden units?

Procedure

Fifteen minutes in your group, write the answers in the notebook; then plenum, one question at a time. Nothing is computed in Case 1.

Case 1: what we hope you say (a–b)

Case 1: what we hope you say (a–b)

a)

The expected error, on data drawn from the same distribution and never seen in training, of the *procedure* "this architecture with these hyperparameters, trained this way on $n(k-1)/k$ points" – averaged over which points those are. It is about the procedure because the k fold models are different networks (different training sets, same recipe) that are averaged and discarded; the network you keep is a new fit on all n points, which CV never evaluated.

Case 1: what we hope you say (a–b)

a)

The expected error, on data drawn from the same distribution and never seen in training, of the *procedure* "this architecture with these hyperparameters, trained this way on $n(k-1)/k$ points" – averaged over which points those are. It is about the procedure because the k fold models are different networks (different training sets, same recipe) that are averaged and discarded; the network you keep is a new fit on all n points, which CV never evaluated.

b)

$\bar{e} = \frac{1}{k}\sum e_j$, $s^2 = \frac{1}{k-1}\sum(e_j - \bar{e})^2$, standard error $s/\sqrt{k}$ (3.93). Two training sets each omit one fold and share the other $k-2$ of the $k-1$ folds they contain: a fraction $(k-2)/(k-1)$ in common, 3/4 for $k=5$. Positively correlated fold errors have a larger variance of the mean than independent ones, so $s/\sqrt{k}$ is *optimistic*. $k=2$: training sets of $n/2$, a pessimistic estimate but nearly independent folds; $k=n$: no size bias and no fold randomness, but n almost identical training sets, highly correlated errors, high variance, and n fits.

Case 1: what we hope you say (c–d)

Case 1: what we hope you say (c–d)

c)

Fitted on all the data, the scaler has seen the mean and the standard deviation of the held-out fold, so information about the validation points enters the training – the held-out fold is no longer "future data". The numbers that leak: the fold's mean and variance of x (and of y if the targets are scaled). A `Pipeline` refits *every* step on the training part of each fold when the library calls `fit`, and uses the stored statistics when it calls `predict` on the held-out part.

Case 1: what we hope you say (c–d)

c)

Fitted on all the data, the scaler has seen the mean and the standard deviation of the held-out fold, so information about the validation points enters the training – the held-out fold is no longer "future data". The numbers that leak: the fold's mean and variance of x (and of y if the targets are scaled). A `Pipeline` refits *every* step on the training part of each fold when the library calls `fit`, and uses the stored statistics when it calls `predict` on the held-out part.

d)

First, the learning rate was *chosen on the test set*: the test MSE was minimised over ten candidates, so 0.0094 is the minimum of ten noisy numbers and no longer estimates the error of anything. Second, there is no uncertainty: ten single numbers, no standard error. Honest version: "Of ten learning rates, five-fold cross-validation on the training data selected $\gamma = 0.01$ with a CV MSE of $x \pm s$ (training MSE y); the network refitted at $\gamma = 0.01$ on all training data has a test MSE of z, computed once."

Case 1: what we hope you say (e–f)

Case 1: what we hope you say (e–f)

e)

The number of epochs changes the fitted function exactly as λ or the width does – a network trained for 100 epochs and one trained for 3000 are different models – and it cannot be chosen by the training error, which is non-increasing in the epochs by construction. Hold out a validation fold, record its error after every epoch, take the epoch at which it is smallest, and train the final model for that many epochs (or keep the weights of that epoch). Averaged over folds this is a validation curve over epochs.

Case 1: what we hope you say (e–f)

e)

The number of epochs changes the fitted function exactly as λ or the width does – a network trained for 100 epochs and one trained for 3000 are different models – and it cannot be chosen by the training error, which is non-increasing in the epochs by construction. Hold out a validation fold, record its error after every epoch, take the epoch at which it is smallest, and train the final model for that many epochs (or keep the weights of that epoch). Averaged over folds this is a validation curve over epochs.

f)

$\lambda_{\mathbf{1SE}} = \max\{\lambda : \mathrm{CV}(\lambda) \le \mathrm{CV}(\lambda_{\min}) + \mathrm{SE}(\lambda_{\min})\}$: the largest penalty whose CV error is within one standard error of the best. The models in that set cannot be told apart by the data, so the improvement of the less regularised one is not believed, and the simpler model has lower variance. Over hidden units "more regularised" means *fewer* units – the smallest network within one standard error of the best; over epochs, the fewest epochs.

Case 2: your network inside `scikit-learn`'s cross-validation

The data and the folds

The data cell of Project 2 part b), verbatim: $n = 100$, x uniform on $[-1, 1]$, $y = 1/(1 + 25x^2) + \varepsilon$, $\sigma = 0.1$, seed 2026. One `KFold(5, shuffle=True, random_state=2026)` object, used for *everything* below so that every comparison is on the same folds.

Step 1: the wrapper

Fill in `fit` and `predict` around your network (the three rules of Part A.16). *Checkpoint:* `fit` on all 100 points, `predict` on them, and the training MSE agrees with your own loop to every digit.

Step 2: a first look

`cross_val_score` on the pipeline `StandardScaler` $\rightarrow$ `net`: five negative MSEs; mean $\pm$ $s/\sqrt{5}$. `cross_validate` for the training MSEs beside them. *Checkpoint:* the training MSE is below the CV MSE, both above $\sigma^2 = 0.01$.

Step 3: the grid

`GridSearchCV` over $\gamma \in \{0.001, \ldots, 0.1\} \times \lambda \in \{0, 10^{-5}, \ldots, 10^{-2}\}$; from `cv_results_`: heat map, standard errors (`std_test_score`$/\sqrt{k-1}$), the one-standard-error choice.

Case 2: steps 4–6

Step 4 (optional): validation curves

`validation_curve` over the hidden units $\{5, 10, 25, 50, 100\}$ and over the epochs $\{50, \ldots, 3000\}$: CV and training MSE against the value. *Checkpoint:* the training curve only falls – where does the CV curve stop falling? Optional in exercise 2 and Project 2; same machinery as step 3.

Step 5: the benchmark

OLS and Ridge polynomials of degree 1–15 on the *same* cv: one table with CV MSE $\pm$ s.e., parameters and `mean_fit_time` for the polynomial and the network.

Step 6: the test set, once

A fresh set of 100 points (seed 1); `best_estimator_` – the winner refit on all 100 training points – is evaluated on it once, and so is the polynomial. No going back.

Numbers to write down

The five fold errors; the best grid point with its error bar and the one-SE choice; the test MSE of network and polynomial – these go into Project 2. If you did step 4: where the CV curves stop falling. Each fit takes under a second, the grid about a minute.

Case 2: the code you should end up with (steps 1–2)

```
class Week41Regressor(BaseEstimator, RegressorMixin):
    def __init__(self, ...): # as on the Part A.16 frame
        ...
    def fit(self, X, y):
        rng = np.random.default_rng(self.seed)
        sizes = [X.shape[1], *self.hidden, 1]
        layers = create_layers(sizes, self.activation, rng)
        self.layers_, self.history_ = train(
            X, y.reshape(-1, 1), layers, self.activation,
            "mse", gamma=self.gamma, epochs=self.epochs,
            batch_size=self.batch_size, lmbd2=self.lmbd,
            rng=rng)
        return self
    def predict(self, X):
        out = predict(X, self.layers_, self.activation, "mse")
        return out[:, 0]

cv = KFold(n_splits=5, shuffle=True, random_state=2026)
net = make_pipeline(StandardScaler(), Week41Regressor())
mse = "neg_mean_squared_error"
scores = -cross_val_score(net, X, y, cv=cv, scoring=mse)
print(scores.mean(), scores.std(ddof=1) / np.sqrt(5))
res = cross_validate(net, X, y, cv=cv, scoring=mse,
                     return_train_score=True)
```

- `fit` builds the layers from the seed and calls the week-41 `train`; `predict` runs the forward pass and flattens to 1D.
- `y.reshape(-1, 1)`: the week-41 code wants a column; the library hands over a 1D array.
- Every fold starts from `clone(net)`: a fresh scaler and a fresh, untrained network with the same seed.
- `scores` are negative MSEs: flip the sign.

Case 2: the code you should end up with (steps 3–4)

```
grid = {"week41regressor__gamma":
            [0.001, 0.003, 0.01, 0.03, 0.1],
        "week41regressor__lmbd":
            [0.0, 1e-5, 1e-4, 1e-3, 1e-2]}
search = GridSearchCV(net, grid, cv=cv, scoring=mse,
                      return_train_score=True).fit(X, y)
r = search.cv_results_
cvmse = -r["mean_test_score"].reshape(5, 5)
se = r["std_test_score"].reshape(5, 5) / np.sqrt(5 - 1)
best = search.best_index_  # ddof=0 !
within = cvmse <= cvmse.min() + se.ravel()[best]

tr, va = validation_curve(
    net, X, y, cv=cv, scoring=mse,
    param_name="week41regressor__hidden",
    param_range=[(5,), (10,), (25,), (50,), (100,)])
# tr, va: (5 values, 5 folds); the same for "..._epochs\"
```

- `week41regressor__gamma`: step name, two underscores, parameter name.
- `cv_results_` is a dictionary of arrays, one entry per grid point, in the order of `params`; reshape to the grid.
- `std_test_score` is the population standard deviation (`ddof=0`), hence the $\sqrt{k-1}$.
- `validation_curve` is one-dimensional grid search that hands back the training scores too.

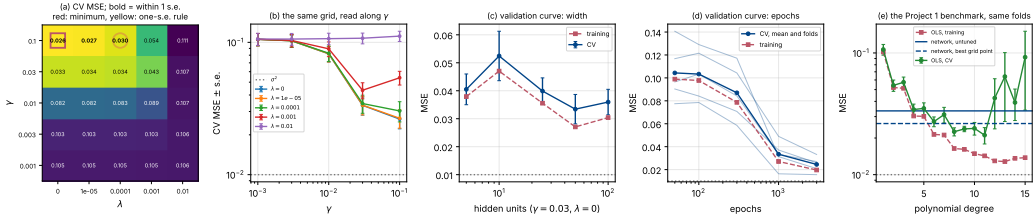
Case 2: the code you should end up with (steps 5–6)

```
poly = make_pipeline(StandardScaler(), PolynomialFeatures(),
                     LinearRegression())
ols = GridSearchCV(poly,
       {"polynomialfeatures__degree": range(1, 16)},
       cv=cv, scoring=mse).fit(X, y)
# the same with Ridge(alpha=...) for LinearRegression()

X_test, y_test = fresh_data(seed=1) # once, at the end
pred = search.best_estimator_.predict(X_test)
mse_test = np.mean((pred - y_test) ** 2)
pred = ols.best_estimator_.predict(X_test)
mse_ols = np.mean((pred - y_test) ** 2)
```

- The polynomial runs on the *same* `cv`: that is what makes the comparison fair.
- `best_estimator_` is the whole pipeline – scaler and network – refit on all 100 points.
- The test set is generated (or unpacked) here and nowhere earlier; it is used once, for every model, and nothing is changed afterwards.

Case 2: what you should find (`week44_cv.pdf`, seed 2026)



(a) The 5×5 grid; bold cells are within one standard error of the minimum (red); the one-standard-error choice (yellow) is the largest λ among them. (b) The same numbers read along γ. (c)–(d) Validation curves over width and epochs with the training curves beside them (the optional step 4). (e) OLS polynomials on the same folds against the network before and after the grid.

Case 2: the numbers behind the figure (steps 2–4)

Step 2: the untuned 1–50–1 network

Folds 0.031, 0.049, 0.016, 0.033, 0.037; CV MSE 0.0334 ± 0.0053, training 0.0271. Above $\sigma^2 = 0.01$, and the folds differ by a factor of three – why one number without an error bar means little.

Step 3: the grid

Minimum at $\gamma = 0.1$, $\lambda = 0$: 0.0263 ± 0.0041; three cells within one standard error; the one-SE rule picks $\gamma = 0.1$, $\lambda = 10^{-4}$ (0.0303) – a weak preference, since $\lambda \leq 10^{-4}$ changes nothing the data can see. The learning rate matters most: at $\gamma = 0.001$ the network has not converged in 1000 epochs (0.105, the variance of y), at $\gamma = 0.1$ it has. The penalty does nothing until 10^{-2}, where it hurts.

Step 4: the curves

Width: CV MSE $0.041, 0.052, 0.040, 0.033, 0.036$ for $5, 10, 25, 50, 100$ units with standard errors 0.005–0.009: no trend the error bars resolve – at this γ and 1000 epochs the width is not what limits the fit. Epochs: $0.104, 0.103, 0.087, 0.033, 0.025$ for $50, \ldots, 3000$ – still falling at 3000 with $\gamma = 0.03$: under-trained, not over-trained; early stopping would not stop it yet.

Case 2: the numbers behind the figure (steps 5–6)

Step 5: the benchmark

OLS: degree 11, CV MSE 0.0211 ± 0.0032; Ridge: degree 11, $\lambda = 10^{-4}$, 0.0210. The untuned network (0.033) is worse than the polynomial; the tuned one (0.026) is within one standard error of it – with 151 parameters against 12 and 0.6 s per fit against milliseconds.

Step 6: the test set, once

Network `best_estimator_`: 0.0148; OLS: 0.0133; Ridge: 0.0135. All three are *below* their CV scores (0.026, 0.021) – the opposite of what the optimism of Part A.14 predicts. Two effects compete: the refit on 100 points instead of 80 (more data, and 25% more SGD steps per epoch) pulls the error down; the selection of the minimum pushes the CV score down. Here the first wins; on another test draw it might not. One test number has no error bar either.

What goes into Project 2

The (γ, λ) heat map with the one-SE cell marked, the benchmark table, *one* test number per model, and a sentence on how each hyperparameter was chosen. The validation curves over width and epochs only if you did the optional step 4.

Case 2: do you have the insight?

1. `GridSearchCV` refits the best point on all 100 points. Why is the CV error of that point *not* the error you should expect from `best_estimator_` – in which direction is it off, and for two different reasons?
2. The validation curve over epochs is still falling at 3000. Does that mean more epochs are always better for this network? What would you do before concluding?
3. The polynomial of degree 11 and the tuned network are within one standard error of each other. Which do you report as "better" in Project 2, and what sentence goes with the number?
4. Someone runs the grid with `cv=KFold(5)` without `shuffle=True`. The data cell sorted x. What goes wrong?

Case 2: do you have the insight?

1. `GridSearchCV` refits the best point on all 100 points. Why is the CV error of that point *not* the error you should expect from `best_estimator_` – in which direction is it off, and for two different reasons?
2. The validation curve over epochs is still falling at 3000. Does that mean more epochs are always better for this network? What would you do before concluding?
3. The polynomial of degree 11 and the tuned network are within one standard error of each other. Which do you report as "better" in Project 2, and what sentence goes with the number?
4. Someone runs the grid with `cv=KFold(5)` without `shuffle=True`. The data cell sorted x. What goes wrong?

What we hope you say

(1) The CV score is pessimistic because the fold models had 80 points instead of 100, and optimistic because it is the minimum of 25 noisy estimates; which wins depends on the problem – here the test MSE came out *below* the CV score, so the first did. (2) No: at $\gamma = 0.03$ the optimiser is slow and the curve says "not converged"; raise γ (the grid did) or add momentum, then look again – a validation curve over epochs is a statement about *this* optimiser at *this* learning rate. (3) Neither: report both with their error bars and say they cannot be told apart on 100 points; prefer the polynomial on simplicity and speed unless the network wins elsewhere. (4) The folds are contiguous intervals of x: every fold model extrapolates to a region it never saw, the CV error is pessimistic and the chosen hyperparameters are the ones that extrapolate least badly, not the ones that fit best. Shuffle.

Only a mention: the same with PyTorch and Keras

Nothing changes in the theory, and `KFold` only makes indices

```
for train_idx, val_idx in KFold(5, shuffle=True, random_state=2026).split(X):
    scaler = StandardScaler().fit(X[train_idx]) # inside the fold
    net = build_torch_model() # a fresh model
    train(net, scaler.transform(X[train_idx]), y[train_idx])
    errors.append(mse(net, scaler.transform(X[val_idx]), y[val_idx]))
```

Ten lines around any training function, the Monday CNN included; the discipline – scaler inside, fresh model per fold, test set once – is yours to keep, since nothing enforces it.

Or make the framework model a `scikit-learn` estimator

`skorch.NeuralNetRegressor` wraps a PyTorch module, `scikeras.wrappers.KerasRegressor` a Keras model; both obey the three rules, so `GridSearchCV` and `Pipeline` work as in Case 2.

Built-in validation, which is a single split

Keras: `model.fit(X, y, validation_split=0.2, callbacks=[EarlyStopping(patience=10, restore_best_weights=True)])`; PyTorch: `random_split` and the loop by hand. Cheaper than k folds, no standard error; the warnings carry over (the split is the *last* fraction, unshuffled; scale inside it; the test set is still touched once). None of this is needed for Project 2.

Wrap-up, the exercises, and next week

Before Friday (exercises week 44, deadline October 30)

Exercise 1: the six questions, written out. Exercise 2: Case 2 on your own network – the wrapper, the (γ, λ) grid with the heat map and the one-SE cell, the benchmark on the same folds, the test set once; the validation curves over width and epochs (2c–d) are optional. Exercise 3 (optional): the MNIST classifier with `StratifiedKFold`, `cross_val_predict` and the out-of-fold confusion matrix. Exercise 4 (optional): the loop written by hand, checked against the library – the solution is in the notebook. Declare your use of LLMs.

Project 2 (deadline November 6)

Required: γ and λ (L2/L1) chosen by cross-validation – today's heat map and table; cross-validating depth, nodes, activation and epochs is optional. Part e) is the confusion matrix of week 43. Every test number comes from a model whose choices were made on training data alone – say how.

Three sentences to take home

The training error chooses nothing. The cross-validation error chooses the hyperparameters and comes with an error bar that is too small. The test error is computed once and chooses nothing either.

Next Monday, week 45

Chapter 11, recurrent neural networks, and time for your Project 2 questions. Reading for today: Sections 2.12, 3.14, 8.12; the `scikit-learn` user guide (cross-validation, developing estimators).